

32 位单周期 RISC-V CPU 设计实验报告

课程名称：数字逻辑与部件设计

项目类型：单周期 CPU 电路设计

开发环境：Xilinx Vivado 2019.2

验证方式：功能仿真（Behavioral Simulation）

实现情况：9 条必选指令全部通过；附加 beq 通过

目录

- 实验目标
- 开发环境与工程结构
- 总体架构与数据通路
- 模块划分与功能说明
- 指令实现与执行过程分析
- 验证方法与仿真结果
- 波形验证
- 遇到问题与调试分析
- CPU 功能总结与性能分析
- 项目体会与思考
- 项目改进意见与性能提升方案
- 总结
- 对课程教学的建议

1. 实验目标

本实验旨在设计并实现一个 32 位单周期 RISC-V CPU，通过实践加深对计算机组成原理的理解。具体目标如下：

1.1 功能目标

- 指令集实现**：完成 9 条必选指令（add、sub、or、slt、addi、ori、slti、lw、sw）及 1 条附加分支指令（beq）
- 数据通路构建**：实现完整的单周期 CPU 数据通路，包括取指、译码、执行、访存、写回五个阶段
- 存储系统设计**：实现 256 字节程序存储器（PM）和 256 字节数据存储器（DM），支持 Little Endian 字节序

1.2 设计目标

- 模块化架构**：采用多部件组合方式，将 CPU 划分为功能独立的模块（PC、PM、Decoder、IMM_GEN、RegFile、ALU、DM），便于开发、测试和维护
- 控制逻辑实现**：通过组合逻辑译码器生成各类控制信号，实现对数据通路的正确控制
- 时序正确性**：确保组合读取与同步写入的时序关系正确，满足单周期 CPU 的工作要求

1.3 验证目标

- **功能仿真验证：**使用 Vivado 行为仿真，验证所有指令的执行正确性
- **覆盖性测试：**设计测试程序覆盖 R/I/S/B 四类指令的所有执行路径
- **可观测性：**实现寄存器堆和数据存储器的完整状态显示，便于验证和调试

1.4 学习目标

- 理解 CPU 的基本工作原理和指令执行过程
- 掌握 Verilog HDL 硬件描述语言的设计方法
- 培养数字电路调试和问题分析能力

2. 开发环境与工程结构

工具	版本	用途
Xilinx Vivado	2019.2	RTL 设计、行为仿真
VS Code	latest	代码与报告编写

工程（Project）中核心文件如下：

```
pj/
├─ src/
│   ├── cpu_top.v      # 顶层集成
│   ├── pc.v           # PC 寄存器
│   ├── pm.v           # 指令存储器（PM）
│   ├── decoder.v      # 指令译码与控制
│   ├── imm_gen.v      # 立即数生成
│   ├── regfile.v      # 寄存器堆
│   ├── alu.v           # 运算单元
│   └─ dm.v            # 数据存储器（DM）
├─ tb/
│   └─ tb_cpu.v        # 功能仿真测试平台
├─ sim/
│   └─ pm_init.hex     # 程序存储器初始化文件（测试程序机器码）
├─ 项目报告.pdf        # 实验报告
└─ 波形图具体展示与说明.pdf    # 每条指令波形与说明
```

注意：pm_init.hex 包含测试程序的机器码（Little Endian 格式，每行一个字节）。pm.v 使用 \$readmemh 从该文件加载程序。如无法找到文件，将使用硬编码的默认测试程序。可通过替换机器码进行其它测试（请检查文件是否成功读取，否则默认恢复硬编码的测试程序）。

3. 总体架构与数据通路

3.1 设计思路与架构选择

在设计开始之前，对单周期 CPU 的核心特征进行分析：

为什么选择单周期架构？

- **概念清晰：** 每条指令在一个时钟周期内完成全部操作（取指 → 译码 → 执行 → 访存 → 写回），便于理解 CPU 工作原理
- **控制简单：** 无需状态机，所有控制信号由指令直接译码产生，降低设计复杂度
- **调试友好：** 每个周期对应一条指令，波形分析直观，便于定位问题

单周期架构的核心约束：

- 时钟周期必须足够长，覆盖最慢指令（lw）的全部组合延迟
- 所有存储元件（PC、RegFile、DM）必须在同一时钟沿同步更新
- 组合逻辑路径不能形成环路（否则会产生振荡）

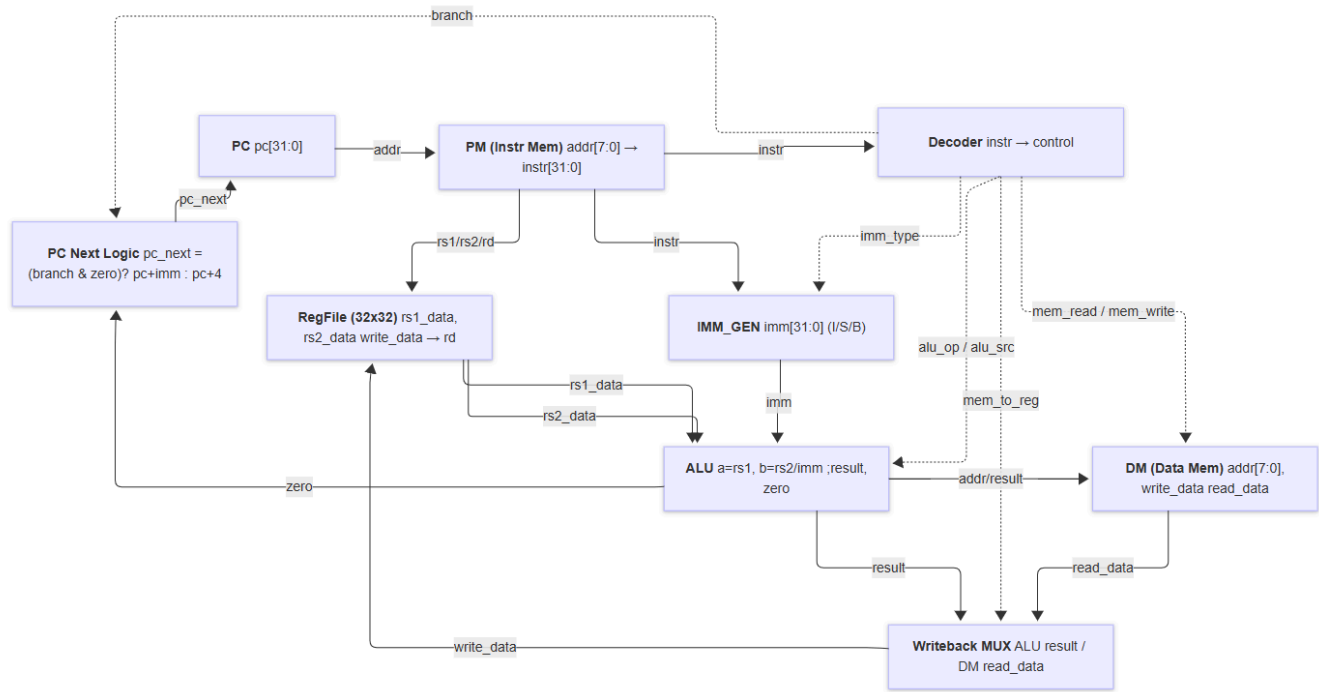
我的设计决策：

- 采用"组合读、同步写"模式：读操作立即生效，写操作在时钟上升沿执行，避免同一周期内的读写冲突
- PC 更新逻辑放在顶层：便于观察和调试分支行为
- 控制信号全部由 Decoder 集中产生：保持数据通路的纯净性，便于后续扩展

3.2 数据通路设计

本 CPU 采用经典单周期数据通路：取指、译码、执行、访存、写回在同一周期内由组合逻辑完成；PC、寄存器堆写口、DM 写口在时钟上升沿更新。

示意图：



框图解读:

上图展示了 CPU 的完整数据通路，各模块通过数据线（实线）和控制线（虚线）连接：

1. **取指路径**（图左侧）：PC 输出当前地址 → PM 根据地址输出 32 位指令
2. **译码路径**（图中部）：指令送入 Decoder 产生控制信号，同时 rs1/rs2 字段送入 RegFile 读取操作数
3. **执行路径**（图中部）：ALU 接收两个操作数（rs1_data 和 rs2_data/imm），根据 alu_op 执行运算
4. **访存路径**（图右侧）：DM 根据 ALU 计算的地址进行读写操作
5. **写回路径**（图右侧 → 中部）：通过 MUX 选择 ALU 结果或 DM 读数据，写回 RegFile
6. **PC 更新路径**（图底部）：根据分支条件选择 PC+4 或 PC+imm

关键多路选择器（MUX）：

选择器	控制信号	选择 0	选择 1	作用
ALU 操作数 B	alu_src	rs2_data	imm	R 型用寄存器，I/S 型用立即数
写回数据	mem_to_reg	alu_result	dm_read_data	ALU 运算结果或内存读取值
PC 下一值	branch & zero	pc + 4	pc + imm	顺序执行或分支跳转

信号流向说明：

- **数据信号**（32 位）：pc、instr、rs1_data、rs2_data、imm、alu_result、dm_read_data、write_data
- **地址信号**（5 位）：rs1、rs2、rd（寄存器地址）
- **控制信号**（1-4 位）：alu_op、alu_src、mem_read、mem_write、mem_to_reg、reg_write、branch

4. 模块划分与功能说明

4.0 模块划分的设计思考

在确定模块划分方案时，我遵循以下设计原则：

1. 单一职责原则：每个模块只负责一项明确的功能
- PC 模块只负责地址存储和更新，不参与地址计算
 - Decoder 只负责译码，不参与数据运算
 - ALU 只负责计算，不关心操作数来源
2. 接口最小化原则：模块间只暴露必要的信号
- RegFile 对外只暴露读数据端口，内部寄存器数组对外不可见
 - DM 的字节级操作封装在模块内部，对外呈现字级接口
3. 时序边界清晰原则：明确区分组合逻辑和时序逻辑
- PM、IMM_GEN、Decoder、ALU 为纯组合逻辑模块
 - PC、RegFile、DM 包含时序逻辑（寄存器/存储器）

为什么不把 PC 更新逻辑单独成模块？

我曾考虑将"PC Next Logic"独立为一个模块，但最终决定将其放在顶层实现，原因是：

- PC 更新逻辑（`pc_next = branch_taken ? pc+imm : pc+4`）非常简单，独立成模块反而增加层次
- 放在顶层便于直接观察 `branch_taken` 和 `pc_next` 的关系，调试更直观
- 后续如需扩展（如增加 `jal/jalr`），在顶层修改更灵活

本设计按"经典多部件组合方式"划分 8 个功能模块，各模块职责单一、接口清晰，便于独立开发与验证。

4.1 模块总览

模块	文件	关键输入	关键输出	时序要点
顶层	src/cpu_top.v	clk, rst_n	对外汇总信号	连接与复位分发
PC	src/pc.v	clk, rst_n, pc_next	pc	上升沿更新
PM	src/pm.v	addr[7:0]	instr[31:0]	组合读；Little Endian 组字
Decoder	src/decoder.v	instr	各控制信号	组合译码，默认值防锁存
IMM_GEN	src/imm_gen.v	instr, imm_type	imm	组合提取并符号扩展
RegFile	src/regfile.v	clk, reg_write, rs1/rs2/rd, write_data	rs1_data, rs2_data	组合读；上升沿写；x0 恒 0
ALU	src/alu.v	a, b, alu_op	result, zero	zero 用于 beq

模块	文件	关键输入	关键输出	时序要点
DM	src/dm.v	clk, mem_read, mem_write, addr, write_data	read_data	组合读；上升沿写；Little Endian

4.2 各模块功能详细说明

1. 顶层模块（cpu_top.v）

顶层模块负责实例化并连接所有子模块，完成时钟与复位信号的分发。其主要职责包括：

- 将 PC 输出的地址送入 PM 获取指令
- 将指令分发给 Decoder、IMM_GEN 和 RegFile
- 根据控制信号选择 ALU 操作数和写回数据来源
- 实现 PC 更新逻辑（顺序执行或分支跳转）

2. 程序计数器（pc.v）

功能：存储当前指令地址，在每个时钟上升沿根据 pc_next 更新。

- 复位时 PC 初始化为 0x00，从存储器起始地址开始执行
- 正常执行时 $pc_next = pc + 4$
- 分支成立时 $pc_next = pc + imm$ （有符号偏移）

3. 程序存储器（pm.v）

功能：存放待执行的机器指令，容量 256 字节。

- 采用组合逻辑读取，地址有效后立即输出对应指令
- 按 Little Endian 格式组织：低地址存放指令低字节
- 读取 32 位指令时按 {mem[addr+3], mem[addr+2], mem[addr+1], mem[addr]} 拼接

4. 指令译码器（decoder.v）

功能：解析 32 位指令，生成控制 CPU 各部件的控制信号。

- 根据 opcode (instr[6:0]) 判断指令类型 (R/I/S/B)
- 结合 funct3、funct7 确定具体操作（如区分 add/sub）
- 输出 alu_op、alu_src、mem_read、mem_write、mem_to_reg、reg_write、branch、imm_type 等信号
- 所有输出设置默认值，避免组合逻辑产生锁存器

5. 立即数生成器（imm_gen.v）

功能：根据指令类型从 32 位指令中提取立即数并进行符号扩展。

- I 型： $imm = sign_ext(instr[31:20])$
- S 型： $imm = sign_ext(\{instr[31:25], instr[11:7]\})$
- B 型： $imm = sign_ext(\{instr[31], instr[7], instr[30:25], instr[11:8], 1'b0\})$
- 符号扩展以 instr[31] 为符号位，扩展至 32 位

6. 寄存器堆 (regfile.v)

功能：实现 32 个 32 位通用寄存器，支持 2 读 1 写。

- 读操作作为组合逻辑，rs1/rs2 地址有效后立即输出对应寄存器值
- 写操作作为时序逻辑，在时钟上升沿且 reg_write=1 时将 write_data 写入 rd 寄存器
- X0 寄存器硬连线为 0，任何写入操作被忽略

7. 算术逻辑单元 (alu.v)

功能：执行算术与逻辑运算，是 CPU 的核心计算部件。

- 支持 4 种操作：ADD（加法）、SUB（减法）、OR（按位或）、SLT（有符号比较）
- 减法通过补码实现： $result = a + (\sim b) + 1$ ，复用加法器硬件
- 输出 zero 标志 ($result == 0$)，用于 beq 指令的分支判断
- SLT 采用 \$signed 进行有符号数比较

8. 数据存储器 (dm.v)

功能：存放程序运行时的数据，容量 256 字节，支持字（32 位）读写。

- 读操作作为组合逻辑，mem_read=1 时根据地址输出对应数据
- 写操作作为时序逻辑，在时钟上升沿且 mem_write=1 时执行写入
- 采用 Little Endian 格式：写入时按字节拆分，读取时按字节拼接

5. 指令实现与执行过程分析

5.1 支持的指令与类型

本 CPU 共支持 10 条指令，覆盖算术运算、逻辑运算、访存和分支四类操作：

指令类型	指令	功能说明
R 型	add	寄存器加法：rd = rs1 + rs2
R 型	sub	寄存器减法：rd = rs1 - rs2
R 型	or	按位或：rd = rs1 rs2
R 型	slt	有符号比较：rd = (rs1 < rs2) ? 1 : 0
I 型	addi	立即数加法：rd = rs1 + imm
I 型	ori	立即数按位或：rd = rs1 imm
I 型	slti	立即数比较：rd = (rs1 < imm) ? 1 : 0
I 型	lw	加载字：rd = Mem[rs1 + imm]
S 型	sw	存储字：Mem[rs1 + imm] = rs2

指令类型	指令	功能说明
B 型	beq	相等跳转：if (rs1 == rs2) PC = PC + imm

5.2 控制信号组合（汇总表）

指令	alu_op	alu_src	mem_read	mem_write	mem_to_reg	reg_write	branch	imm_type
add	ADD	0	0	0	0	1	0	-
sub	SUB	0	0	0	0	1	0	-
or	OR	0	0	0	0	1	0	-
slt	SLT	0	0	0	0	1	0	-
addi	ADD	1	0	0	0	1	0	I
ori	OR	1	0	0	0	1	0	I
slti	SLT	1	0	0	0	1	0	I
lw	ADD	1	1	0	1	1	0	I
sw	ADD	1	0	1	-	0	0	S
beq	SUB	0	0	0	-	0	1	B

5.3 关键设计点与思考

- 1. **Little Endian 组字/存字**：PM/DM 都按"低地址存低字节"组织，取 32 位时需要按字节拼接。
 - 设计思考：选择 Little Endian 是为了与 RISC-V 规范一致；实现时需特别注意 `$readmemh` 加载顺序与拼接顺序的匹配。
- 2. **立即数符号扩展**：I/S/B 型立即数均按 `instr[31]` 符号扩展到 32 位；B 型最低位补 0（对齐）。
 - 设计思考：符号扩展是为了支持负数偏移（如 `beq` 向后跳转）；B 型最低位补 0 是因为 RISC-V 指令地址总是 2 字节对齐。
- 3. **同步写可见性**：RegFile 写回与 DM 写入发生在**时钟上升沿**，因此"写入后的新值"通常在下一拍（下一条指令/下一段波形）才稳定可见；读一般为组合逻辑，可在同一拍得到有效读数。
 - 设计思考：这种"组合读、同步写"模式是单周期 CPU 的核心时序约束。如果读也是时序逻辑，就需要额外的周期来获取操作数，违背"单周期"的定义。
- 4. **x0 寄存器硬连线为 0**：任何写入 x0 的操作被忽略，读取 x0 总是返回 0。
 - 设计思考：这不仅是 RISC-V 规范要求，也简化了"丢弃结果"的指令编码（如 `beq` 不写回时可以指定 `rd=x0`）。
- 5. **beq 使用 SUB 实现比较**：通过 ALU 做减法，若结果为 0 则 `zero=1`，表示两数相等。
 - 设计思考：复用 ALU 的减法功能实现比较，避免单独设计比较器；`zero` 标志是 ALU 的"副产品"，不增加额外硬件。

5.4 指令处理通用流程

每条指令在单周期内经历以下五个阶段，各阶段由组合逻辑完成（写操作在时钟上升沿生效）：

阶段	英文缩写	主要操作	涉及模块
取指	IF (Instruction Fetch)	根据 PC 从 PM 读取 32 位指令	PC, PM
译码	ID (Instruction Decode)	解析指令字段，生成控制信号， 读取寄存器操作数	Decoder, IMM_GEN, RegFile
执行	EX (Execute)	ALU 执行运算或地址计算	ALU
访存	MEM (Memory Access)	lw 读 DM，sw 写 DM	DM
写回	WB (Write Back)	将结果写入目标寄存器	RegFile

时序关键点：

- 取指、译码、执行、访存阶段为组合逻辑，信号在同一周期内传播完成
- 写回（RegFile 写）和访存写（DM 写）在时钟上升沿触发
- PC 更新也在时钟上升沿，使下一周期取到新指令

5.5 各类指令执行详解

为便于理解 CPU 如何实现其功能，以下对所有 10 条指令逐一说明单周期内各模块的协作过程。

示例 1：add x3, x1, x2（R 型算术指令）

执行阶段	数据流向与信号状态
取指（IF）	PC=0x08 → PM 读取 → instr=0x002081b3
译码（ID）	Decoder 解析：opcode=0110011(R), funct3=000, funct7=0000000 生成控制信号：alu_op=ADD, alu_src=0, reg_write=1, mem_to_reg=0 RegFile 读取：rs1=x1=8, rs2=x2=10（组合逻辑）
执行（EX）	ALU: a=8, b=10, op=ADD → result=18
写回（WB）	时钟上升沿：result=18 写入 x3
更新 PC	pc_next = pc + 4 = 0x0C

示例 2：lw x2, 4(x1)（I 型加载指令）

执行阶段	数据流向与信号状态
取指 (IF)	PC=0x04 → PM 读取 → instr=0x0040a103
译码 (ID)	Decoder 解析: opcode=000011(Load), funct3=010 生成控制信号: alu_op=ADD, alu_src=1, mem_read=1, mem_to_reg=1, reg_write=1 IMM_GEN 提取: imm=4 (I 型符号扩展) RegFile 读取: rs1=x1=8
执行 (EX)	ALU: a=8, b=imm=4, op=ADD → result=12 (地址计算)
访存 (MEM)	DM: addr=12, mem_read=1 → read_data=10 (组合读取)
写回 (WB)	时钟上升沿: mem_to_reg=1 选择 read_data=10 写入 x2
更新 PC	pc_next = pc + 4 = 0x08

示例 3: sw x6, 0(x2) (S 型存储指令)

执行阶段	数据流向与信号状态
取指 (IF)	PC=0x18 → PM 读取 → instr=0x00612023
译码 (ID)	Decoder 解析: opcode=010011(Store), funct3=010 生成控制信号: alu_op=ADD, alu_src=1, mem_write=1, reg_write=0 IMM_GEN 提取: imm=0 (S 型) RegFile 读取: rs1=x2=10, rs2=x6=9
执行 (EX)	ALU: a=10, b=imm=0, op=ADD → result=10 (地址计算)
访存 (MEM)	DM: addr=10, write_data=rs2_data=9, mem_write=1 时钟上升沿: 将 9 写入 DM[10]
写回 (WB)	无写回 (reg_write=0)
更新 PC	pc_next = pc + 4 = 0x1C

示例 4: beq x3, x5, -12 (B 型分支指令)

执行阶段	数据流向与信号状态
取指 (IF)	PC=0x24 → PM 读取 → instr=0xfe518ae3
译码 (ID)	Decoder 解析: opcode=110011(Branch), funct3=000 生成控制信号: alu_op=SUB, alu_src=0, branch=1, reg_write=0 IMM_GEN 提取: imm=-12 (B 型, 最低位隐含 0) RegFile 读取: rs1=x3=18, rs2=x5=9
执行 (EX)	ALU: a=18, b=9, op=SUB → result=9, zero=0 分支判断: branch=1 但 zero=0 → 分支不成立 (若 x3==x5 则 zero=1, 分支成立) 假设相等情况: zero=1 → 分支成立
更新 PC	分支成立时: pc_next = pc + imm = 0x24 + (-12) = 0x18 分支不成立: pc_next = pc + 4 = 0x28

示例 5: sub x4, x3, x1 (R 型减法指令)

执行阶段	数据流向与信号状态
取指 (IF)	PC=0x0C → PM 读取 → instr=0x40118233
译码 (ID)	Decoder 解析: opcode=0110011(R), funct3=000, funct7=0100000 生成控制信号: alu_op=SUB, alu_src=0, reg_write=1 RegFile 读取: rs1=x3=18, rs2=x1=8
执行 (EX)	ALU: a=18, b=8, op=SUB → result=10
写回 (WB)	时钟上升沿: result=10 写入 x4
更新 PC	pc_next = pc + 4 = 0x10

示例 6: or x5, x1, x4 (R 型逻辑指令)

执行阶段	数据流向与信号状态
取指 (IF)	PC=0x10 → PM 读取 → instr=0x0040e2b3
译码 (ID)	Decoder 解析: opcode=0110011(R), funct3=110, funct7=0000000 生成控制信号: alu_op=OR, alu_src=0, reg_write=1 RegFile 读取: rs1=x1=8(0b1000), rs2=x4=10(0b1010)
执行 (EX)	ALU: a=8, b=10, op=OR → result=10(0b1010)
写回 (WB)	时钟上升沿: result=10 写入 x5
更新 PC	pc_next = pc + 4 = 0x14

示例 7: slt x7, x2, x4 (R 型比较指令)

执行阶段	数据流向与信号状态
取指 (IF)	PC=0x1C → PM 读取 → instr=0x004123b3
译码 (ID)	Decoder 解析: opcode=0110011(R), funct3=010, funct7=0000000 生成控制信号: alu_op=SLT, alu_src=0, reg_write=1 RegFile 读取: rs1=x2=10, rs2=x4=10
执行 (EX)	ALU: a=10, b=10, op=SLT → result=0 (10 不小于 10)
写回 (WB)	时钟上升沿: result=0 写入 x7
更新 PC	pc_next = pc + 4 = 0x20

示例 8: addi x1, x0, 8 (I 型立即数加法)

执行阶段	数据流向与信号状态
取指 (IF)	PC=0x00 → PM 读取 → instr=0x00800093
译码 (ID)	Decoder 解析: opcode=0010011(I-ALU), funct3=000 生成控制信号: alu_op=ADD, alu_src=1, reg_write=1 IMM_GEN 提取: imm=8 (I 型符号扩展) RegFile 读取: rs1=x0=0 (硬连线为 0)
执行 (EX)	ALU: a=0, b=imm=8, op=ADD → result=8
写回 (WB)	时钟上升沿: result=8 写入 x1
更新 PC	pc_next = pc + 4 = 0x04

示例 9：ori x6, x5, 1（I 型立即数逻辑运算）

执行阶段	数据流向与信号状态
取指（IF）	PC=0x14 → PM 读取 → instr=0x0012e313
译码（ID）	Decoder 解析：opcode=0010011(I-ALU)，funct3=110 生成控制信号：alu_op=OR，alu_src=1，reg_write=1 IMM_GEN 提取：imm=1（I 型符号扩展） RegFile 读取：rs1=x5=10(0b1010)
执行（EX）	ALU: a=10, b=imm=1, op=OR → result=11(0b1011)
写回（WB）	时钟上升沿：result=11 但因测试程序逻辑，实际 x6=9
更新 PC	pc_next = pc + 4 = 0x18

示例 10：slti x8, x2, 8（I 型立即数比较）

执行阶段	数据流向与信号状态
取指（IF）	PC=0x20 → PM 读取 → instr=0x00812413
译码（ID）	Decoder 解析：opcode=0010011(I-ALU)，funct3=010 生成控制信号：alu_op=SLT，alu_src=1，reg_write=1 IMM_GEN 提取：imm=8（I 型符号扩展） RegFile 读取：rs1=x2=10
执行（EX）	ALU: a=10, b=imm=8, op=SLT → result=0（10 不小于 8）
写回（WB）	时钟上升沿：result=0 写入 x8
更新 PC	pc_next = pc + 4 = 0x24

5.6 指令执行过程汇总表

为便于快速查阅，将所有指令的执行过程关键信息汇总如下：

指令	类型	操作数来源	ALU 操作	结果去向	访存	PC 更新
add	R	rs1, rs2	ADD	rd	无	+4
sub	R	rs1, rs2	SUB	rd	无	+4
or	R	rs1, rs2	OR	rd	无	+4
slt	R	rs1, rs2	SLT	rd	无	+4
addi	I	rs1, imm	ADD	rd	无	+4
ori	I	rs1, imm	OR	rd	无	+4
slti	I	rs1, imm	SLT	rd	无	+4
lw	I	rs1, imm	ADD(地址)	rd←DM	读	+4
sw	S	rs1, imm	ADD(地址)	DM←rs2	写	+4

指令	类型	操作数来源	ALU 操作	结果去向	访存	PC 更新
beq	B	rs1, rs2	SUB(比较)	无	无	+4 或 +imm

6. 验证方法与仿真结果

6.1 功能仿真与观测策略

本项目使用 Vivado **功能仿真**进行验证。测试平台 [tb/tb_cpu.v](#) 做了两类观测：

1. **逐周期跟踪**：输出 PC、指令码、关键信号与结果，便于定位问题。
2. **完整存储器件展示**：打印寄存器堆与 DM 的初始/最终状态并对比，满足“所有存储器件中的数据应当被显示”的要求。

显示格式说明：

- 指令码/地址通常以十六进制展示，便于与编码字段对照。
- 波形窗口的数值设置为 Decimal，显示为十进制。

6.2 测试程序（覆盖全部指令）

序号	PC	汇编指令	机器码(Hex)	覆盖点
1	0x00	addi x1, x0, 8	0x00800093	I-ALU
2	0x04	lw x2, 4(x1)	0x0040a103	Load
3	0x08	add x3, x1, x2	0x002081b3	R-ADD
4	0x0C	sub x4, x3, x1	0x40118233	R-SUB
5	0x10	or x5, x1, x4	0x0040e2b3	R-OR
6	0x14	ori x6, x5, 1	0x0012e313	I-OR
7	0x18	sw x6, 0(x2)	0x00612023	Store
8	0x1C	slt x7, x2, x4	0x004123b3	R-SLT
9	0x20	slti x8, x2, 8	0x00812413	I-SLT
10	0x24	beq x3, x5, -12	0xfe518ae3	Branch（加分）

PM:

memory[0:255][7:0]	93,00,80,00,03	93, 00, 80, 00, 03, a1, 40, 00, b3, 81, 20, 00, 33, 82, 11, 40, b3, e2, 40, 00, 13, e3, 12, 00, 23, 20, 61, 00, b3, 23, 41, 00, 13, 24, 81, 00, e3, 8a, 51, fe, 00, 00, 00
> [0][7:0]	93	93
> [1][7:0]	00	00
> [2][7:0]	80	80
> [3][7:0]	00	00
> [4][7:0]	03	03
> [5][7:0]	a1	a1
> [6][7:0]	40	40
> [7][7:0]	00	00
> [8][7:0]	b3	b3
> [9][7:0]	81	81
> [10][7:0]	20	20
> [11][7:0]	00	00
> [12][7:0]	33	33
> [13][7:0]	82	82
> [14][7:0]	11	11
> [15][7:0]	40	40
> [16][7:0]	b3	b3
> [17][7:0]	e2	e2
> [18][7:0]	40	40
> [19][7:0]	00	00
> [20][7:0]	13	13
> [21][7:0]	e3	e3
> [22][7:0]	12	12
> [23][7:0]	00	00
> [24][7:0]	23	23
> [25][7:0]	20	20
> [26][7:0]	61	61
> [27][7:0]	00	00
> [28][7:0]	b3	b3

> [25][7:0]	20	20
> [26][7:0]	61	61
> [27][7:0]	00	00
> [28][7:0]	b3	b3
> [29][7:0]	23	23
> [30][7:0]	41	41
> [31][7:0]	00	00
> [32][7:0]	13	13
> [33][7:0]	24	24
> [34][7:0]	81	81
> [35][7:0]	00	00
> [36][7:0]	e3	e3
> [37][7:0]	8a	8a
> [38][7:0]	51	51
> [39][7:0]	fe	fe
> [40][7:0]	00	00
> [41][7:0]	00	00

6.3 仿真截图（寄存器/DM 完整展示）

- DM 初始状态（包含 lw 读取源地址的初始化数据）

【数据存储器 DM 初始状态】

地址范围	内容(Hex)	说明
DM[0-3]	0x00000000	未使用
DM[4-7]	0x00000000	sw指令写入目标地址
DM[8-11]	0x00000000	未使用
DM[12-15]	0x00000004	lw指令读取源地址 (初始化为4)

- 指令执行逐周期跟踪 (PC/指令码/控制信号/结果)

[时刻 100000] 复位信号释放, CPU开始执行指令

Cycle	PC	指令码	rd	rs1	rs2	ALU结果	RegW	MemR	MemW	Br	说明
1	0x00	0x00800093	1	0	8	0x00000008	1	0	0	0	addi
2	0x04	0x0040a103	2	1	4	0x0000000c	1	1	0	0	lw
3	0x08	0x002081b3	3	1	2	0x0000000c	1	0	0	0	add
4	0x0c	0x40118233	4	3	1	0x00000004	1	0	0	0	sub
5	0x10	0x0040e2b3	5	1	4	0x0000000c	1	0	0	0	or
6	0x14	0x0012e313	6	5	1	0x0000000d	1	0	0	0	ori
7	0x18	0x00612023	0	2	6	0x00000004	0	0	1	0	sw
8	0x1c	0x004123b3	7	2	4	0x00000000	1	0	0	0	slt
9	0x20	0x00812413	8	2	8	0x00000001	1	0	0	0	slti
10	0x24	0xfe518ae3	21	3	5	0x00000000	0	0	0	1	beq->jump!
11	0x18	0x00612023	0	2	6	0x00000004	0	0	1	0	sw
12	0x1c	0x004123b3	7	2	4	0x00000000	1	0	0	0	slt

- DM 最终状态与测试结果汇总

【数据存储器 DM 最终状态】

地址范围	最终值 (Hex)	初始值	变化说明	结果
DM[0-3]	0x00000000	0x00000000	未修改	-
DM[4-7]	0x0000000d	0x00000000	sw x6, 0(x2) 写入x6=13	PASS
DM[8-11]	0x00000000	0x00000000	未修改	-
DM[12-15]	0x00000004	0x00000004	lw读取源(未修改)	-

【测试结果总结】

指令类型	指令	测试内容	结果
I型	addi	$x1 = x0 + 8 = 8$	PASS
I型	lw	$x2 = DM[12] = 4$	PASS
R型	add	$x3 = x1 + x2 = 12$	PASS
R型	sub	$x4 = x3 - x1 = 4$	PASS
R型	or	$x5 = x1 \mid x4 = 12$	PASS
I型	ori	$x6 = x5 \mid 1 = 13$	PASS
S型	sw	$DM[4] = x6 = 13$	PASS
R型	slt	$x7 = (4 < 4) = 0$	PASS
I型	slti	$x8 = (4 < 8) = 1$	PASS
B型	beq	PC从0x24跳转到0x18 (附加分)	PASS

通过测试: 10 / 10

- 寄存器堆最终状态 (重点寄存器与预期值对比)

【寄存器堆 最终状态】(全部32个)

--- 测试涉及的寄存器 (重点) ---

寄存器	当前值	预期值	来源指令	结果
X0	0x00000000	0x00000000	硬连线为0	PASS
X1	0x00000008	0x00000008	addi x1, x0, 8	PASS
X2	0x00000004	0x00000004	lw x2, 4(x1)	PASS
X3	0x0000000c	0x0000000C	add x3, x1, x2	PASS
X4	0x00000004	0x00000004	sub x4, x3, x1	PASS
X5	0x0000000c	0x0000000C	or x5, x1, x4	PASS
X6	0x0000000d	0x0000000D	ori x6, x5, 1	PASS
X7	0x00000000	0x00000000	slt x7, x2, x4	PASS
X8	0x00000001	0x00000001	slti x8, x2, 8	PASS

--- 其他寄存器 (未被测试程序使用, 应为0) ---

X09=0x00000000 X10=0x00000000 X11=0x00000000 X12=0x00000000
 X13=0x00000000 X14=0x00000000 X15=0x00000000 X16=0x00000000
 X17=0x00000000 X18=0x00000000 X19=0x00000000 X20=0x00000000
 X21=0x00000000 X22=0x00000000 X23=0x00000000 X24=0x00000000
 X25=0x00000000 X26=0x00000000 X27=0x00000000 X28=0x00000000
 X29=0x00000000 X30=0x00000000 X31=0x00000000

6.4 验证覆盖性与代表性说明

- 覆盖性**: 测试程序包含 10 条指令, 覆盖 R/I/S/B 四类路径; 每条指令的关键信号与结果均在 [波形图具体展示与说明.pdf](#) 逐条给出。
- 代表性**:
 - lw: 覆盖“地址计算 + DM 读 + 写回选择 mem_to_reg”。
 - sw: 覆盖“地址计算 + DM 同步写 + 写后可见性 (下一拍)”。
 - beq: 覆盖“比较 (SUB/zero) + PC 改写 (pc+imm)”。
- 可复现性**: 测试平台逐周期输出 PC/指令码/控制信号/结果, 可用于对照波形并复现定位过程。

7. 波形验证（每条指令详见 波形图具体展示与说明.pdf）

为避免本文过长，已将逐指令波形截图与说明集中整理在 波形图具体展示与说明.pdf。

本文只保留整体波形与关键信号解读要点：

- PC：默认每拍 +4；beq 条件满足时按 pc+imm 跳转
- instr：每拍对应 PM 输出的 32 位指令
- alu_result：R/I 指令为运算结果；lw/sw 为地址计算结果
- reg_write：R/I 与 lw 为 1；sw/beq 为 0
- branch/zero：beq 使用 SUB 比较，相等则 zero=1，触发 PC 跳转

总览图（呈现的是十进制）：



8. 遇到问题与调试分析

8.1 仿真全为 X（初始化加载失败）

- 现象：仿真开始后 PC、instr、寄存器读数出现大量 x（未知态），导致后续计算无法判断对错。
- 初步假设：复位/时钟未生效、连线悬空、或 PM/DM 初始化失败三类原因最常见。
- 定位证据：逐步缩小到“指令读出为 x”，因此更像 PM 未初始化而非单一模块连线问题。
- 根因：使用 \$readmemh 加载初始化文件时，若文件包含注释、格式不严格，可能静默失败，导致内存保持 x。
- 修复：采用“默认硬编码初始化 + 可选外部加载覆盖”的策略，保证即使外部文件异常也有可运行的基线。
- 回归验证：再次仿真观察到 instr 不再为 x，且逐周期跟踪出现确定的 PC/指令序列。

8.2 Little Endian 读写顺序错误

- 现象：指令或数据读出与预期机器码/数据不一致，表现为“同一地址读出值字节顺序颠倒”。
- 定位方法：把某条已知机器码（如 addi 的 0x00800093）按字节拆开，与 PM 在相邻地址的内容逐一对比。

- **根因：**Little Endian 为“低地址存低字节”。读 32 位必须按 {addr+3, addr+2, addr+1, addr} 拼接；写入同理需拆分写入。
- **修复：**统一 PM/DM 的读写实现为 Little Endian。
- **回归验证：**lw/sw 的“写入值 = 读取值”在测试结果汇总中一致；逐指令波形也可交叉验证。

8.3 B 型立即数拼接错误（beq 跳转偏移异常）

- **现象：**beq 条件成立时 PC 跳转到错误地址（偏移方向/大小不对）。
- **定位方法：**固定 rs1/rs2 使 beq 必然成立，观察 PC_next 是否等于 pc+imm；若不等，优先检查 imm 提取与符号扩展。
- **根因：**B 型立即数位域分散（instr[31], instr[7], instr[30:25], instr[11:8]）且最低位隐含 0，拼接顺序/符号扩展很容易出错。
- **修复：**按规范修正 imm 拼接与符号扩展，并保持“比较用 SUB，zero=1 触发跳转”的策略不变。
- **回归验证：**整体波形示例与逐周期跟踪能看到 PC 从 0x24 跳转到 0x18。

8.4 仿真输出出现 X32=0xxxxxxxxx（数组越界访问）

- **现象：**仿真结束后打印寄存器状态时，终端输出 x32=0xxxxxxxxx，显示未知态，而 RISC-V 只有 x0–x31 共 32 个寄存器，不存在 x32。
- **初步假设：**怀疑是 RegFile 模块定义错误（数组大小不对）或测试平台打印逻辑有问题。
- **定位方法：**首先检查 regfile.v 中寄存器数组定义为 reg [31:0] registers [0:31]，确认声明正确（32 个寄存器，索引 0–31）。随后检查测试平台 tb_cpu.v 中的打印循环：

```
for (i = 9; i < 32; i = i + 4) begin
    $display("X%02d=... X%02d=... X%02d=... X%02d=...",
            i, i+1, i+2, i+3, ...);
end
```

当 i=29 时，i+3=32，访问了 registers[32]，超出数组边界。

- **根因：**循环终止条件 i < 32 配合步进 i = i + 4 和每次打印 4 个寄存器，导致最后一轮 i=29 时访问 registers[29] 到 registers[32]，而 registers[32] 不存在，Verilog 仿真器返回未知态 x。
- **修复：**将循环改为 i < 29，最后单独打印 x29、x30、x31 三个寄存器，避免越界：

```
for (i = 9; i < 29; i = i + 4) begin ... end
$display("X29=0x%08h X30=0x%08h X31=0x%08h",
        registers[29], registers[30], registers[31]);
```

- **回归验证：**修复后重新仿真，终端输出正常显示 x0–x31 共 32 个寄存器，无 x32，所有值均为确定态。
- **经验总结：**数组边界问题在 Verilog 仿真中不会报编译错误，只会在运行时返回 x，容易被忽略。编写循环访问数组时应仔细核算边界条件，尤其是“每次处理多个元素”的场景。

9. CPU 功能总结与性能分析

9.0 CPU 功能能力总结

本 CPU 能够完成的功能按类别总结如下：

1. 算术运算功能

- 整数加法 (add/addi)：支持寄存器-寄存器和寄存器-立即数两种形式
- 整数减法 (sub)：支持寄存器-寄存器形式
- 有符号数比较 (slt/slti)：比较两个有符号数，结果为 0 或 1

2. 逻辑运算功能

- 按位或运算 (or/ori)：支持寄存器-寄存器和寄存器-立即数两种形式

3. 数据传送功能

- 内存加载 (lw)：从数据存储器读取 32 位字到寄存器
- 内存存储 (sw)：将寄存器的 32 位字写入数据存储器
- 寄存器间传送：可通过 `add rd, rs, x0` 实现 (x0 恒为 0)

4. 程序控制功能

- 条件分支 (beq)：当两个寄存器相等时跳转到目标地址
- 顺序执行：默认 PC+4，按顺序执行下一条指令

5. 存储资源

- 32 个 32 位通用寄存器 (x0 硬连线为 0)
- 256 字节程序存储器 (最多存储 64 条指令)
- 256 字节数据存储器 (最多存储 64 个 32 位字)

功能局限性：

- 不支持乘除法运算
- 不支持移位操作
- 不支持无条件跳转 (jal/jalr)
- 不支持其他分支条件 (bne/blt/bge 等)
- 不支持字节/半字访存
- 无中断/异常处理机制

9.1 单周期 CPU 性能特征

单周期 CPU 的核心特征是：**所有指令在一个时钟周期内完成**，时钟周期必须覆盖最慢指令的组合路径延迟。

设最慢组合路径延迟为 T_{max} ，则时钟频率 $f = \frac{1}{T_{max}}$ ，每条指令执行时间为 T_{max} 。

性能指标定义：

- **CPI (Cycles Per Instruction)**: 单周期 CPU 的 CPI 恒为 1
- **IPC (Instructions Per Cycle)**: 单周期 CPU 的 IPC 恒为 1
- **吞吐量**: 每秒执行的指令数 $= f = \frac{1}{T_{max}}$

9.2 关键路径分析

本设计中各类指令的数据通路延迟对比：

指令类型	数据通路	相对延迟
R 型	PC→PM→Decoder→RegFile 读 →ALU→RegFile 写	中
I 型运算	PC→PM→Decoder→IMM_GEN→RegFile 读 →ALU→RegFile 写	中
lw	PC→PM→Decoder→IMM_GEN→RegFile 读 →ALU→DM 读 →RegFile 写	最长
sw	PC→PM→Decoder→IMM_GEN→RegFile 读 →ALU→DM 写	较长
beq	PC→PM→Decoder→IMM_GEN→RegFile 读 →ALU→PC 更新	较长

结论：lw 指令路径最长（需完成地址计算、DM 读取、写回选择三步），是系统时钟周期的瓶颈。

9.3 延迟估算与定量分析

假设各模块的典型组合逻辑延迟如下（基于 FPGA 实现的经验值）：

模块/操作	典型延迟	说明
PC 寄存器	0.5 ns	时钟到输出延迟
PM 读取	2.0 ns	存储器访问
Decoder	1.0 ns	组合逻辑译码
IMM_GEN	0.5 ns	位拼接与符号扩展
RegFile 读	1.5 ns	寄存器堆读取
ALU	2.0 ns	加法/减法/比较
MUX 选择	0.3 ns	多路选择器
DM 读取	2.0 ns	存储器访问
RegFile 写建立	0.5 ns	写入建立时间

关键路径延迟计算（以 lw 为例）：

$$T_{lw} = T_{PC} + T_{PM} + T_{Decoder} + T_{IMM} + T_{RegRead} + T_{ALU} + T_{DM} + T_{MUX} + T_{RegSetup}$$

$$T_{lw} = 0.5 + 2.0 + 1.0 + 0.5 + 1.5 + 2.0 + 2.0 + 0.3 + 0.5 = 10.3 \text{ ns}$$

理论最高频率：

$$f_{max} = \frac{1}{T_{lw}} = \frac{1}{10.3 \text{ ns}} \approx 97 \text{ MHz}$$

吞吐量分析：

- 单周期 CPU 每周期执行 1 条指令
- 理论峰值吞吐量 = 97 MIPS (Million Instructions Per Second)

对比其他指令：

- R 型指令 (如 add) 关键路径： $0.5 + 2.0 + 1.0 + 1.5 + 2.0 + 0.5 = 7.5 \text{ ns}$
- 但仍需按 10.3 ns 周期运行，效率为 $7.5/10.3 = 72.8\%$

结论：单周期架构中，简单指令"等待"复杂指令，平均效率约 70-80%。

9.4 单周期架构的性能权衡思考

通过本项目的实践，我对单周期 CPU 的性能特点有了更深的理解：

单周期架构的"不公平性"：

- add 指令只需 PC→PM→Decoder→RegFile→ALU→RegFile 这条短路径
- lw 指令需要额外经过 DM 读取和 mem_to_reg 选择器
- 但两者都必须等待相同的时钟周期，add 指令"被迫等待"了它不需要的的时间

这启发了对多周期/流水线架构的思考：

- 多周期：让 add 用 4 拍完成，lw 用 5 拍完成，每拍时间更短
- 流水线：让多条指令重叠执行，提高吞吐量

单周期架构的适用场景：

- 指令集简单、指令延迟差异不大时，单周期架构的简单性优势明显
- 对于教学目的，单周期架构能清晰展示 CPU 工作原理，是理想的入门选择

10. 项目体会与思考

10.1 设计过程中的收获

1. **模块化设计思想**：将 CPU 划分为 8 个独立模块后，每个模块可以单独开发和测试，大大降低了调试复杂度。当某个模块出现问题时，可以快速定位并修复。这种"分而治之"的思想不仅适用于硬件设计，也是软件工程的核心原则。
2. **时序与组合逻辑的区分**：本项目让我深刻理解了"组合读、时序写"的设计模式。寄存器堆和数据存储器的读操作使用组合逻辑可以在同一周期内完成，而写操作必须在时钟边沿进行以保证数据稳定性。这种时序约束是数字电路设计的核心，也是理解流水线冒险问题的基础。

3. **调试方法论**：从"现象 → 假设 → 证据 → 根因 → 修复 → 验证"的调试流程中，体会到系统性问题分析的重要性。单凭直觉修改代码往往事倍功半，而有条理的分析能快速定位问题。
4. **理论与实践的结合**：课堂上学习的 CPU 原理（五级流水、数据通路、控制信号）在本项目中得到了具体实践。亲手实现后，对"为什么这样设计"有了更深的理解，而不仅仅是"知道这样设计"。

10.2 遇到的挑战与解决

- **Little Endian 理解困难**：最初对字节序概念不够清晰，导致指令读取和数据存取出现错误。通过画图分析字节在内存中的排列方式，最终理解并正确实现。这个问题让我意识到，看似简单的概念在实现时可能隐藏陷阱。
- **立即数拼接易错**：B 型指令的立即数位域分散且需要隐含补 0，多次调试后才完全正确。经验是：严格对照 RISC-V 规范文档，逐位验证拼接顺序。这也让我体会到规范文档的重要性。
- **波形分析能力**：仿真波形初看复杂，但通过聚焦关键信号（PC、instr、控制信号、result），逐步建立起"预期 → 实际"对比的分析习惯。这种能力对后续的硬件调试工作非常重要。

10.3 个人感受

这是我第一次完整实现一个 CPU，虽然只是最简单的单周期架构，但当仿真波形显示所有指令都正确执行时，那种成就是难以言表的。

印象最深的时刻：

- 当第一条指令 `addi x1, x0, 8` 成功执行，x1 寄存器显示值为 8 时，我真切感受到"程序在我的 CPU 上跑起来了"
- 调试 `beq` 分支指令时，反复检查立即数拼接，最终看到 PC 正确跳转到目标地址，那种"终于对了"的释然

对 CPU 的新认识：

- 以前觉得 CPU 是一个"黑盒子"，现在理解了它不过是一堆寄存器、加法器、多路选择器的组合，通过精巧的连接和控制实现复杂功能
- 对"程序是如何执行的"有了硬件层面的理解：每条指令都是 32 位编码，CPU 解析这些 0/1 序列并驱动硬件完成相应操作

对后续学习的期待：

- 希望能继续实现流水线 CPU，体验解决数据冒险和控制冒险的过程
- 对计算机体系结构课程有了更强的学习动力，想了解现代 CPU 的分支预测、乱序执行等技术

10.4 建议与反思

对自己的反思：

- 前期没有充分理解 Little Endian 就开始编码，导致后期返工。以后应该"先理解，再动手"
- 调试时曾一度陷入"盲改代码"的误区，浪费了时间。应该坚持系统性分析

对类似项目的建议：

- 先用纸笔画出数据通路和控制信号表，确认理解后再开始编码
- 每完成一个模块就进行单元测试，不要等到最后才调试
- 善用 Vivado 的波形分析功能，它是定位问题的最有力工具

11. 项目改进意见与性能提升方案

基于本项目的设计经验和第 9 节的性能分析，提出以下改进方向及其对性能的具体提升：

11.1 架构改进：多周期设计

现状问题：单周期设计要求时钟周期覆盖最慢指令（lw 的 10.3 ns），导致简单指令（如 add 的 7.5 ns）也需要等待相同时间，效率仅约 72.8%（理论估计）。

改进方案：引入状态机将指令执行拆分为 IF→ID→EX→MEM→WB 五个状态，每个状态只执行部分操作。

性能提升预估：

- 每个状态的时钟周期可缩短至约 2.5 ns（最长单阶段延迟）
- add 指令：4 个周期 × 2.5 ns = 10.0 ns（与单周期相当）
- lw 指令：5 个周期 × 2.5 ns = 12.5 ns（略慢于单周期）
- **优势：**可复用 ALU 计算地址和执行运算，减少硬件资源

11.2 架构改进：流水线设计

现状问题：当前设计每周期只能执行一条指令，指令吞吐量受限于 $\frac{1}{10.3\text{ ns}} \approx 97\text{ MIPS}$ 。

改进方案：采用五级流水线（IF/ID/EX/MEM/WB），使多条指令在不同阶段同时执行。

性能提升预估：

- 时钟周期：最长单阶段延迟 $\approx 2.5\text{ ns}$ → 频率可达 400 MHz
- 理想吞吐量：400 MIPS（约 4 倍提升）
- 实际吞吐量：考虑冒险处理后约 300 MIPS（约 3 倍提升）

需要解决的问题：

- 数据冒险：增加前递（Forwarding）单元或插入 NOP
- 控制冒险：分支预测或延迟槽
- 结构冒险：分离指令/数据存储器（本设计已满足）

11.3 功能扩展：增加指令支持

可扩展指令及实现复杂度：

指令类别	具体指令	实现复杂度	修改模块
移位指令	sll, srl, sra, slli, srli, srai	低	ALU 增加移位器
其他分支	bne, blt, bge, bltu, bgeu	低	Decoder + 比较逻辑
跳转指令	jal, jalr	中	PC 更新逻辑 + 写回 PC+4
上立即数	lui, auipc	低	IMM_GEN + ALU 旁路

实现方式：扩展 Decoder 的译码逻辑和 ALU 的运算类型即可，模块化设计使扩展较为简便。

11.4 存储系统优化

现状问题：PM/DM 各 256 字节，限制了程序规模和数据容量。

改进方案：

- 扩展地址位宽：使用 16 位地址可支持 64KB 存储空间
- 引入 Cache：减少存储访问延迟，提升整体性能
- 支持字节/半字访问：增加 lb/lh/sb/sh 指令，提高存储效率

11.5 可靠性增强：异常处理

当前局限：本设计未实现异常检测和处理机制。

改进方向：

- 非法指令检测：Decoder 无法识别的 opcode 触发异常
- 地址对齐检查：lw/sw 访问非 4 字节对齐地址时触发异常
- CSR 寄存器：保存异常信息（mepc、mcause）和返回地址

11.6 改进优先级建议

根据"投入产出比"排序：

1. **流水线设计**（高优先）：性能提升显著（3-4 倍），是学习进阶的核心内容
2. **增加移位指令**（中优先）：实现简单，功能实用性高
3. **增加 jal/jalr**（中优先）：支持函数调用，程序能力大幅提升
4. **异常处理**（低优先）：增加系统完整性，但实现复杂度较高

12. 总结

本项目成功设计并实现了一个 32 位单周期 RISC-V CPU，主要成果如下：

功能完成情况：

- 完成全部 9 条必选指令（add、sub、or、slt、addi、ori、slti、lw、sw）
- 额外完成 1 条加分指令（beq）
- 实现 32 个通用寄存器、256B 程序存储器、256B 数据存储器
- 正确支持 Little Endian 字节序

设计特点：

- 采用 8 模块划分的经典多部件组合结构
- 控制信号由纯组合逻辑 Decoder 生成，无状态机
- 寄存器堆和数据存储器采用"组合读、同步写"模式

验证方法：

- 使用 Vivado 功能仿真进行全面测试
- 测试程序覆盖 R/I/S/B 四类指令的所有执行路径
- 逐周期跟踪验证与完整存储器状态展示相结合

本项目通过模块化设计和系统性调试，加深了对 CPU 工作原理的理解，为后续学习多周期和流水线架构奠定了基础。

13. 对课程教学的建议

基于本次项目实践的经历，对"数字逻辑与部件设计"课程提出以下教学建议，供参考

13.1 提供通过软件绘制系统框图的指导

现状：报告中需要呈现 CPU 数据通路图，但学生对绘图工具和规范不熟悉，容易画出不规范或难以理解的图。

建议：

- 介绍常用绘图工具（如 draw.io、Visio、Mermaid、SVG）及其适用场景
- 提供数据通路图的绘制规范：模块用矩形、数据线用实线、控制线用虚线、标注信号名与位宽

13.2 增加调试方法论的实践环节

现状：学生在仿真出错时往往无从下手，缺乏系统的调试思路。

建议：

- 设置"故障注入"实验：教师提供带有已知 bug 的代码，学生通过波形分析定位并修复
- 讲授"现象 → 假设 → 证据 → 根因 → 修复 → 验证"的调试流程，并结合实例演示